

智能软件开发 方向基础



第六章 集成学习 Ensemble Learning

张朝晖

2023~2024学年第二学期



<https://scikit-learn.org/stable/modules/classes.html#module-sklearn.ensemble>
<https://scikit-learn.org/stable/modules/ensemble.html#ensemble>

<code>ensemble.AdaBoostClassifier([estimator, ...])</code>	An AdaBoost classifier.
<code>ensemble.AdaBoostRegressor([estimator, ...])</code>	An AdaBoost regressor.
<code>ensemble.BaggingClassifier([estimator, ...])</code>	A Bagging classifier.
<code>ensemble.BaggingRegressor([estimator, ...])</code>	A Bagging regressor.

<code>ensemble.RandomForestClassifier([...])</code>	A random forest classifier.
<code>ensemble.RandomForestRegressor([...])</code>	A random forest regressor.
<code>ensemble.RandomTreesEmbedding([...])</code>	An ensemble of totally random trees.
<code>ensemble.StackingClassifier(estimators[, ...])</code>	Stack of estimators with a final classifier.
<code>ensemble.StackingRegressor(estimators[, ...])</code>	Stack of estimators with a final regressor.

序号	内容
1	概述
2	机器学习的基本概念
3	模型的选择与性能评价
4	数据的获取、探索与准备
5	近邻模型-----分类、回归
6	决策树模型-----分类、回归
7	集成学习-----分类、回归
8	(朴素)贝叶斯模型-----分类
9	聚类
10	特征降维及低维可视化(PCA, t-SNE)
11	总复习

主要内容

(以决策树为)个体模型的集成学习

1 Bootstrap Aggregating(bagging)

个体模型是决策树，也可以是其它分类模型

2 Random Forest(RF)

个体模型是决策树

分类—简单投票；

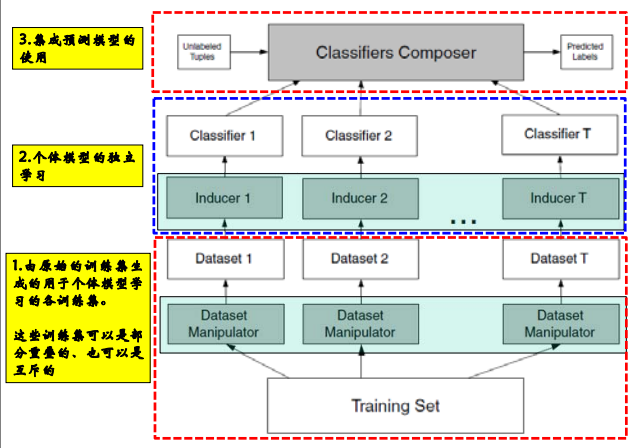
回归—简单平均

3 AdaBoost

4 Stacking



例：面向分类任务的独立个体模型集成



主要内容

1 Bootstrap Aggregating(bagging)

个体模型是决策树，也可以是其它分类模型

2 Random Forest(RF)

个体模型是决策树

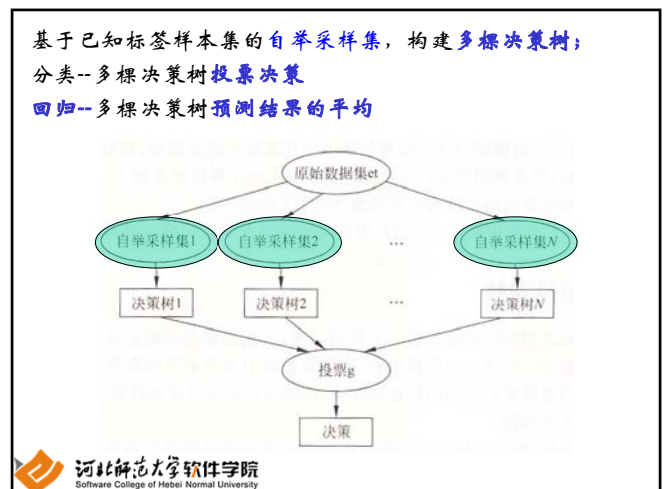
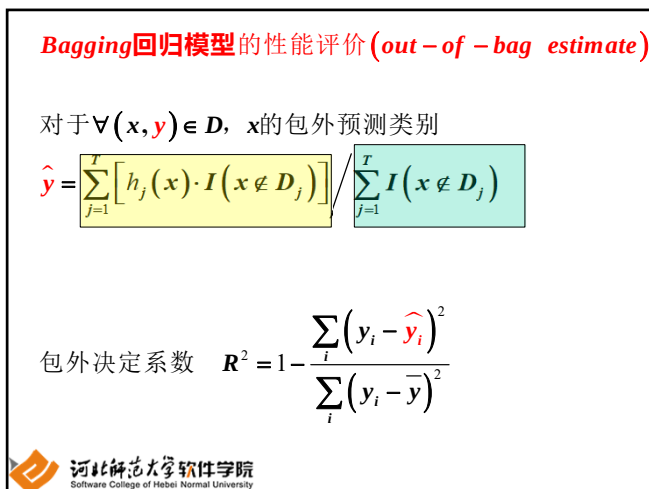
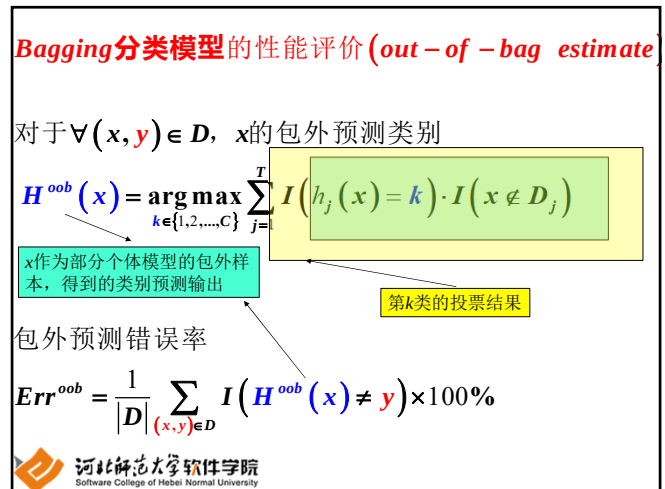
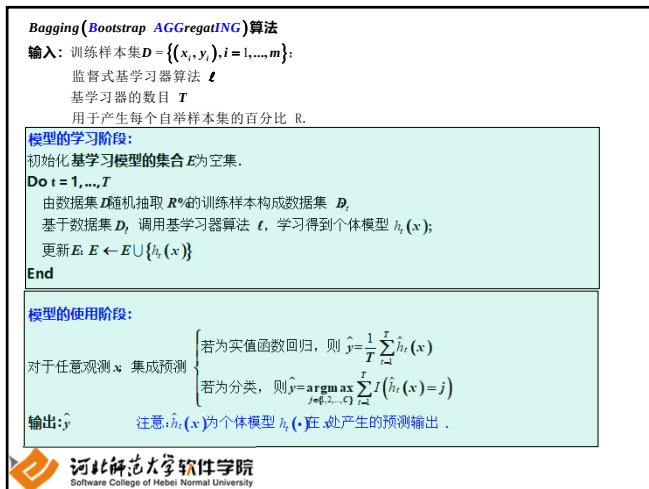
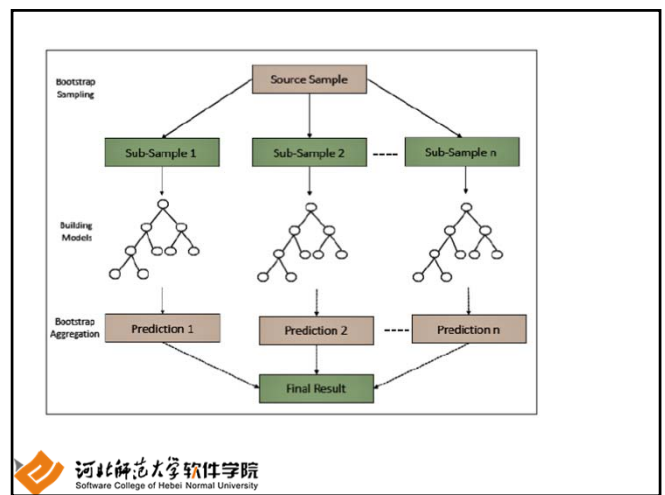
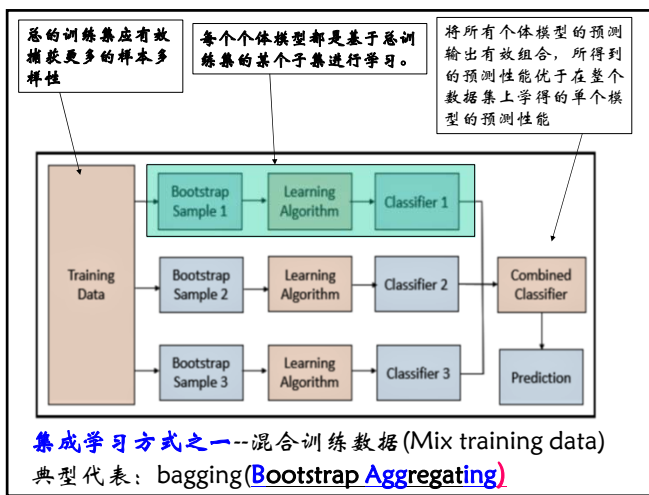
分类—简单投票；

回归—简单平均

3 AdaBoost

4 Stacking





```
class sklearn.ensemble.BaggingClassifier(estimator=None,
n_estimators=10, *, max_samples=1.0, max_features=1.0, bootstrap=True,
bootstrap_features=False, oob_score=False, warm_start=False, n_jobs=None,
random_state=None, verbose=0) [source]
```

```
class sklearn.ensemble.BaggingRegressor(estimator=None,
n_estimators=10, *, max_samples=1.0, max_features=1.0, bootstrap=True,
bootstrap_features=False, oob_score=False, warm_start=False, n_jobs=None,
random_state=None, verbose=0) [source]
```

主要内容

1 Bootstrap Aggregating(bagging)

个体模型是决策树，也可以是其它分类模型

2 Random Forest(RF)

个体模型是决策树

分类—简单投票；

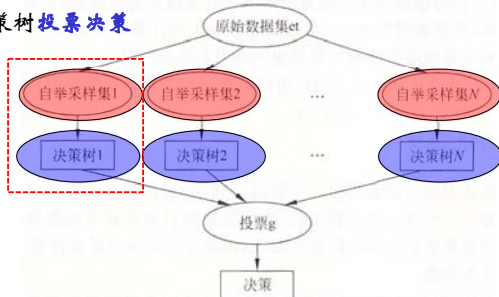
回归—简单平均

3 AdaBoost

4 Stacking

随机森林(bagging + 扰动版的基学习器)

- 基于 **样本集自举重采样** + **特征子集随机抽取**，构建多棵决策树，组成决策树的“森林”；
- 多棵决策树投票决策



2. 基本步骤

关键：样本采样、特征采样，各树彼此独立；投票无偏。

(1)模型学习--构造N棵决策树(不剪枝)。

每棵树的构建，需要：

A--对样本数据进行“自举法(bootstrapping,或自助法)”重采样，得到1个样本集

出发点：使用相同特征空间的不同数据点

B--为该树的每个节点的特征选择，生成备选特征子集。

从特征集内随机抽取m个特征，形成该节点的学习所需要的特征子集。

(2)模型的使用--决策：

输入未知样本，得到多个决策树的输出：

分类--投票，胜者为王；回归--平均，得输出。

Algorithm 2 Random Forests

Let $\mathcal{D} = \{(x_1, y_1), \dots, (x_N, y_N)\}$ denote the training data, with $x_i = (x_{i,1}, \dots, x_{i,p})^T$. For $j = 1$ to J :

- Take a bootstrap sample $\mathcal{D}_j$ of size N from $\mathcal{D}$.
- Using the bootstrap sample $\mathcal{D}_j$ as the training data, fit a tree using binary recursive partitioning
 - Start with all observations in a single node.
 - Repeat the following steps recursively for each unsplit node until the stopping criterion is met:
 - Select m predictors at random from the p available predictors.
 - Find the best binary split among all binary splits on the m predictors from Step (i).
 - Split the node into two descendant nodes using the split from Step (ii).

To make a prediction at a new point x ,

- $\hat{f}(x) = \frac{1}{J} \sum_{j=1}^J \hat{h}_j(x)$ for regression
- $\hat{f}(x) = \arg \max_y \sum_{j=1}^J I(\hat{h}_j(x) = y)$ for classification

where $\hat{h}_j(x)$ is the prediction of the response variable at x using the j th tree (Algorithm 1).

模型的
学习

```
class sklearn.ensemble.RandomForestClassifier(n_estimators=100, *,
criterion='gini', max_depth=None, min_samples_split=2,
min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features='sqrt',
max_leaf_nodes=None, min_impurity_decrease=0.0, bootstrap=True,
oob_score=False, n_jobs=None, random_state=None, verbose=0,
warm_start=False, class_weight=None, ccp_alpha=0.0, max_samples=None,
monotonic_cst=None) [source]
```

```
class sklearn.ensemble.RandomForestRegressor(n_estimators=100, *,
criterion='squared_error', max_depth=None, min_samples_split=2,
min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=1.0,
max_leaf_nodes=None, min_impurity_decrease=0.0, bootstrap=True,
oob_score=False, n_jobs=None, random_state=None, verbose=0,
warm_start=False, ccp_alpha=0.0, max_samples=None,
monotonic_cst=None) [source]
```

主要内容

1 Bootstrap Aggregating(bagging)

个体模型是决策树，也可以是其它分类模型

2 Random Forest(RF)

个体模型是决策树

分类—简单投票

回归—简单平均

3 AdaBoost

4 Stacking

Boosting方法

Boosting 本意——通过增压，加大发动机功率

引申——提升分类器性能

AdaBoost 最为广泛, Adaptive Boosting

主要内容

Boosting方法

Boosting 本意——通过增压，加大发动机功率

引申——提升分类器性能

AdaBoost 最为广泛, Adaptive Boosting

二分类、多分类、实值函数回归

AdaBoost — Adaptive Boosting

Y. Freund and R. E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. Journal of Computer and System Sciences, 55(1):119–139, 1997.

2003年，Schapire 和 Freund 被授予 “the Godel Prize”
-- one of the most prestigious awards in theoretical computer science

基于AdaBoost算法的强分类器训练

输入: (1) 训练样本集 $\mathcal{D} = \{(x_i, y_i) | i = 1, \dots, N\}$

$$y_i \in \{-1, +1\}$$

其中 $\begin{cases} y_i = -1, \text{训练样本 } x_i \text{ 为负样本} \\ y_i = +1, \text{训练样本 } x_i \text{ 为正样本} \end{cases}$

(2) 弱分类器的学习算法 L

(3) 弱分类器的数目 M

注意：此处的弱分类器模型中常用的就是“决策树树桩”，也就是只使用了一个特征生成的决策树。

输出：一个由 M 个弱分类器构成的强分类器

训练过程:

A. 初始化训练样本 x_i 权重 $\mathcal{D}(i)$ $i = 1, \dots, N$

(1) 若正负样本数目一致，则 $\mathcal{D}(i) = \frac{1}{N}$

(2) 若正负样本数目分别为 N_+, N_- ，则 $\begin{cases} \text{正样本 } \mathcal{D}(i) = \frac{1}{2N_+} \\ \text{负样本 } \mathcal{D}(i) = \frac{1}{2N_-} \end{cases}$

B. for $m = 1, \dots, M$

(1) 训练弱分类器 $f_m(x) = L(\mathcal{D}, \mathcal{D}_m) \in \{-1, +1\}$

(2) 估计弱分类器 $f_m(x)$ 的分类错误率 e_m

如: $e_m = \sum_{i=1}^N \mathcal{D}_m(i) I(f_m(x_i) \neq y_i)$ [注: $e_m < 0.5$]

若 $e_m < 0.5$ 则继续(3)与(4)

注意：如果该弱分类器的错误率不是小于0.5，则只是结合其预测情况，更新样本概率分布，但不会参与集成。

B. for $m = 1, \dots, M$ (续前)

(3) 估计弱分类器 $f_m(x)$ 的权重 $c_m = \log \frac{1-e_m}{e_m}$ (注意此处为自然对数)

(4) 基于弱分类器 $f_m(x)$ 调整各样本权重，并归一化

调整: $\mathcal{D}_{m+1}(i) = \mathcal{D}_m(i) \cdot \exp\left[c_m \cdot 1_{f_m(x_i) \neq y_i}\right]$

$$= \begin{cases} \mathcal{D}_m(i) & \text{若 } f_m(x_i) = y_i \\ \mathcal{D}_m(i) \cdot \frac{1-e_m}{e_m} & \text{若 } f_m(x_i) \neq y_i \end{cases}$$

归一化: $\mathcal{D}_{m+1}(i) \leftarrow \frac{\mathcal{D}_{m+1}(i)}{\sum_{j=1}^N \mathcal{D}_{m+1}(j)}$ $i = 1, \dots, N$

C. 强分类器

$$H(x) = \text{sgn}\left[\sum_{m=1}^M c_m f_m(x)\right]$$

“分治策略”调整训练样本的概率分布，使后续弱分类器的学习更关注难度大的训练样本。

若分类器加权组合，得到强分类器。预测性能好的弱分类器在预测中，起较大作用。

Input: Data set $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$;
Base learning algorithm L ;
Number of learning rounds T .

AdaBoost 二分类
模型学习的另一
种描述算法

Process:

1. $\mathcal{D}_1(i) = 1/m$. % Initialize the weight distribution
2. **for** $t = 1, \dots, T$:
3. $h_t = L(D, \mathcal{D}_t)$; % Train a learner h_t from D using distribution $\mathcal{D}_t$
4. $\epsilon_t = \Pr_{x \sim \mathcal{D}_t} [h_t(x) \neq y]$; % Measure the error of h_t
5. **if** $\epsilon_t > 0.5$ **then break**
6. $\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$; % Determine the weight of h_t
7. $\mathcal{D}_{t+1}(i) = \frac{\mathcal{D}_t(i)}{Z_t} \times \begin{cases} \exp(-\alpha_t) & \text{if } h_t(x_i) = y_i \\ \exp(\alpha_t) & \text{if } h_t(x_i) \neq y_i \end{cases}$ % Update the distribution, where
% Z_t is a normalization factor which
% enables $\mathcal{D}_{t+1}$ to be distribution
8. **end**

Output: $H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$

典型应用: real-time face detection



河北师范大学软件学院
Software College of Hebei Normal University

- 1 Bootstrap Aggregating(bagging)
- 2 Random Forest(RF)
- 3 AdaBoost



4. GBDT(Gradient Boosting Decision Tree)
5. XGBoost (eXtreme Gradient Boosting)
6. LightGBM
7. CatBoost(Categorical Boost)

河北师范大学软件学院
Software College of Hebei Normal University

主要内容

1 Bootstrap Aggregating(bagging)

个体模型是决策树，也可以是其它分类模型

2 Random Forest(RF)

个体模型是决策树

分类—简单投票

回归—简单平均

3 AdaBoost

4 Stacking

集成学习模型的其他形式

河北师范大学软件学院
Software College of Hebei Normal University

Stacking 算法描述

输入: 训练集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$;
初级学习算法 $\mathcal{L}_1, \mathcal{L}_2, \dots, \mathcal{L}_T$;
次级学习算法 $\mathcal{L}$.

过程:

- 1: **for** $t = 1, 2, \dots, T$ **do**
- 2: $h_t = \mathcal{L}_t(D)$;
- 3: **end for**
- 4: $D' = \emptyset$;
- 5: **for** $i = 1, 2, \dots, m$ **do**
- 6: **for** $t = 1, 2, \dots, T$ **do**
- 7: $z_{it} = h_t(x_i)$;
- 8: **end for**
- 9: $D' = D' \cup ((z_{i1}, z_{i2}, \dots, z_{iT}), y_i)$;
- 10: **end for**
- 11: $h' = \mathcal{L}(D')$;

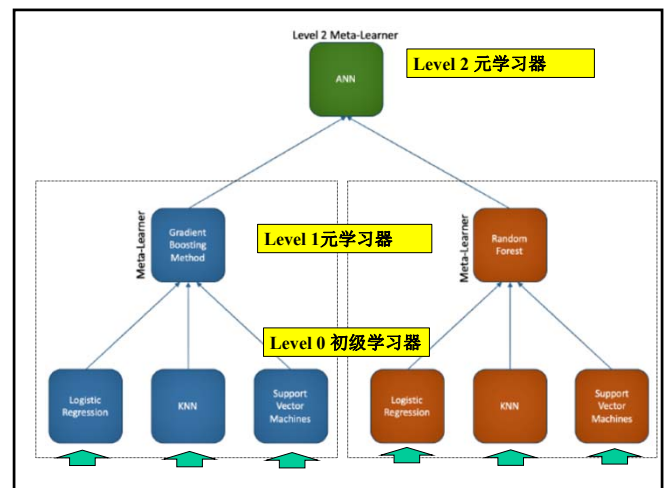
输出: $H(x) = h'(h_1(x), h_2(x), \dots, h_T(x))$

阶段1.利用给定的训练集 D , 调用不同初级学习算法, 学习得到不同个体模型

阶段2.基于多个个体模型预测的新样本集 D' 的构建。

> 针对训练集 D 的每个样本, 利用得到的 T 个个体模型, 对该样本的输出分别进行预测, 以 T 个预测结果作为该样本新的输入向量, 以标签值作为标签, 构成新的训练样本。

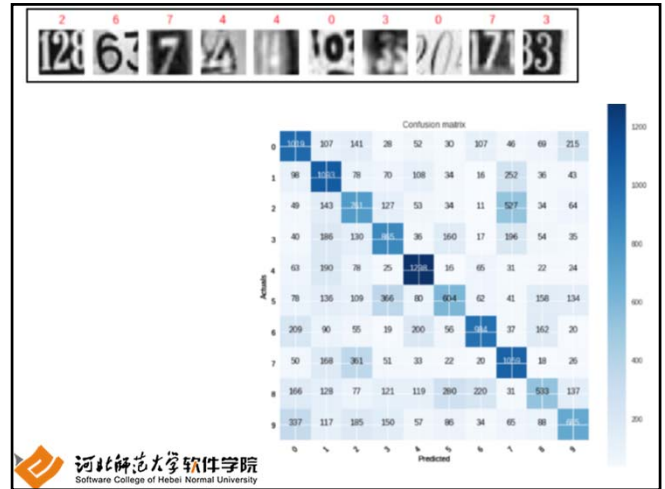
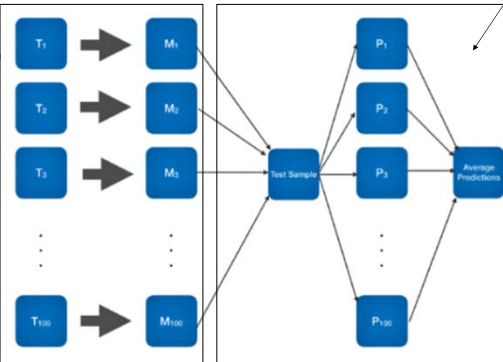
阶段3.调用次级学习算法, 学习得到模型



例：以神经网络为个体模型的街景门牌号码识别

模型的训练：基于不同训练集得到的彼此独立的神经网络模型

模型的使用：每个测试样本分别送入各模型，产生关于所有类别预测概率，计算平均，得到关于每个类别的平均概率；由概率最大，得到相应的预测类别。



思考题

以决策树作为个体模型，采用BAGGING以及Random Forest实现个体模型的并行集成。
给定已知答案的训练集，请对两种集成模型的实现步骤进行详细描述。

- (1) 分类
- (2) 回归

关于特征的重要性评价的讨论

基于置换重要性算法 (permutation importance algorithm) 的特征重要性评分计算

输入：学习得到的预测模型m，用于特征评价的数据集D(训练集或验证集)

STEP1. 计算模型m在数据集D上的参考分值S

例：分类模型的预测正确率、回归模型的决定系数 R^2

STEP2. 对于每个特征 $j \in \{1, 2, \dots, d\}$:

2.1 对于每次随机置换 $k \in \{1, 2, \dots, K\}$:

----随机打乱数据集D中第j个特征的取值，产生该特征取值乱序后的污染数据集 $\bar{D}_{k,j}$

----计算模型m在数据集 $\bar{D}_{k,j}$ 上的预测分值 $s_{k,j}$ 。

2.2 计算特征j的重要性得分:

$$i_j = s - \frac{1}{K} \sum_{k=1}^K s_{k,j}$$

STEP3. 将所有特征重要性得分归一化，得最终评价结果。

https://scikit-learn.org/stable/modules/permutation_importance.html#permutation-importance